Programação 2

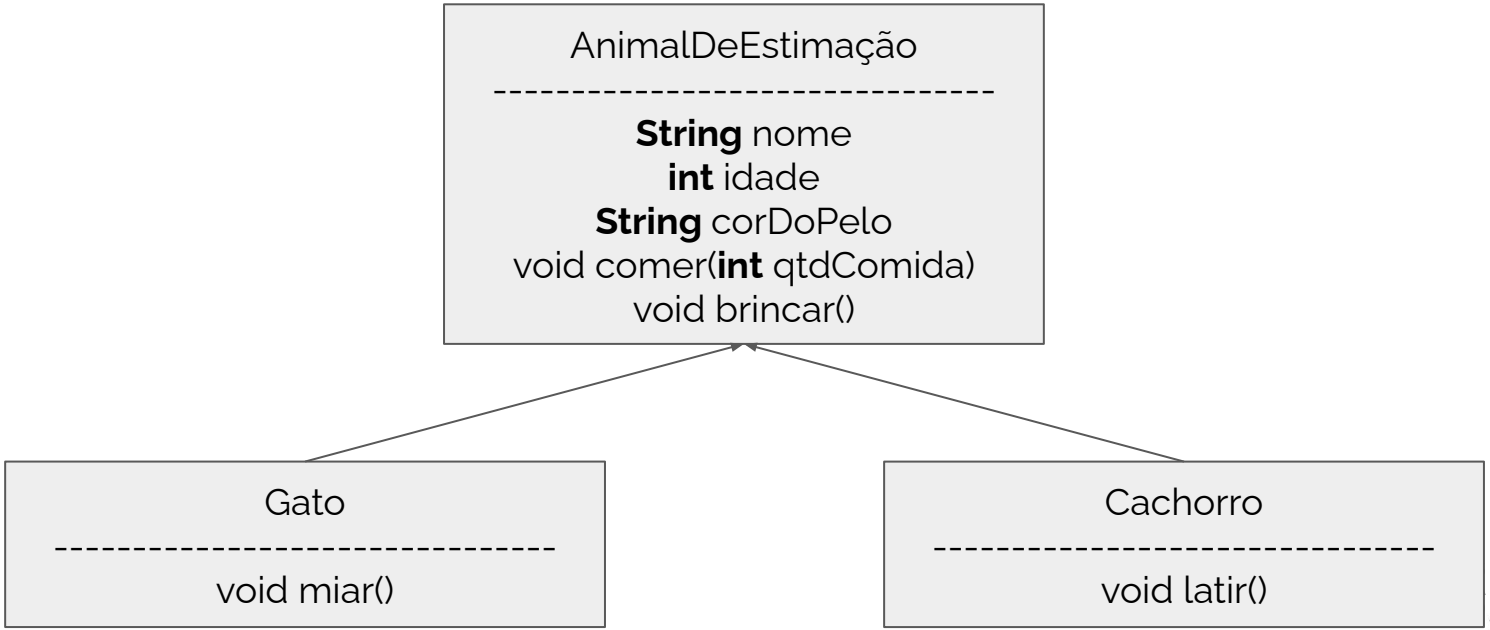
Aula 14 - Arquitetura MVC: interface gráfica, componentes visuais e linguagens de marcação

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Herança

Herança é uma forma de explicitar que uma determinada classe **tem** e **faz** tudo que outra classe faz, e mais um pouco.

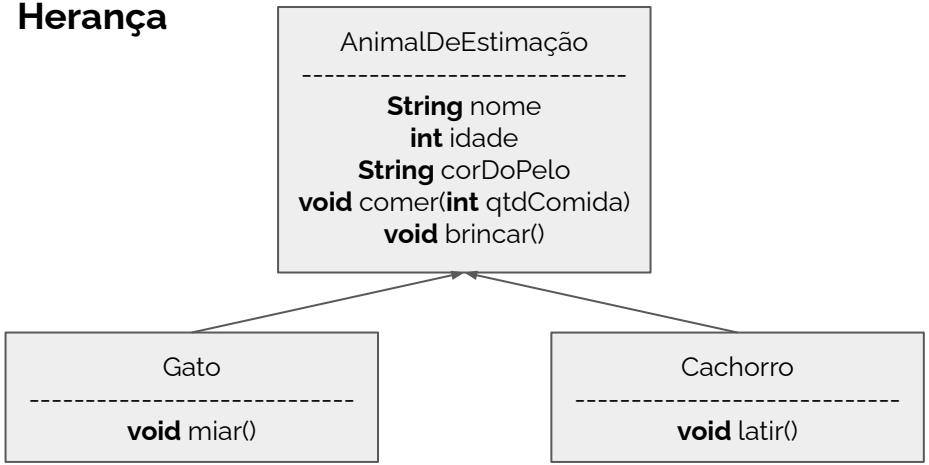


UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

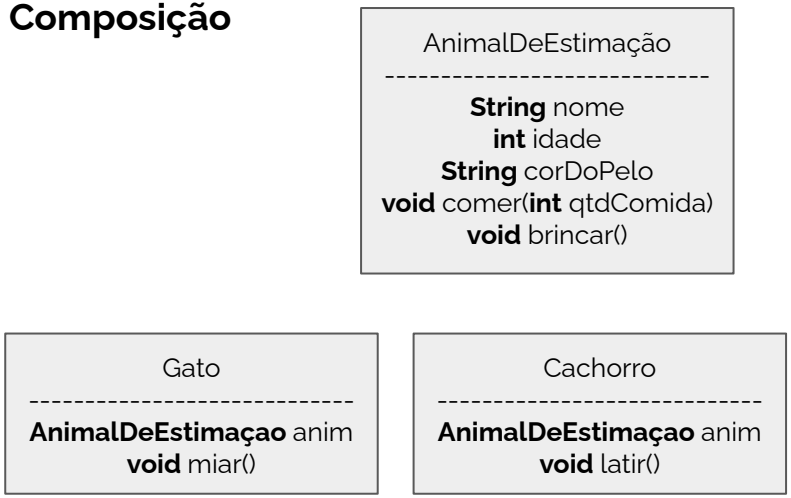
Herança x Composição

Uma opção para resolver o mesmo tipo de problema é usar a composição

Herança



Composição



Herança: SubClasse e SuperClasse

Ao criar uma classe, em vez de declarar membros completamente novos, você pode designar que a nova classe, chamada de **subclasse**,
deva herdar membros de uma classe existente, que é chamada de **superclasse**.

(A linguagem de programação C++ refere-se à superclasse como a classe básica e a subclasse como a classe derivada.) Uma subclasse pode tornar-se uma superclasse para futuras subclasses

Herança: SubClasse

Uma subclasse

- pode adicionar seus próprios campos (**atributos**) e **métodos**, portanto, ela é mais específica que sua superclasse
- exibe os comportamentos da superclasse e **pode modificá-los** de modo que eles operem adequadamente para a subclasse
- é por isso que a herança é às vezes chamada **especialização**.

Herança: Hierarquia

A **superclasse direta** é aquela a partir da qual a subclasse herda explicitamente.

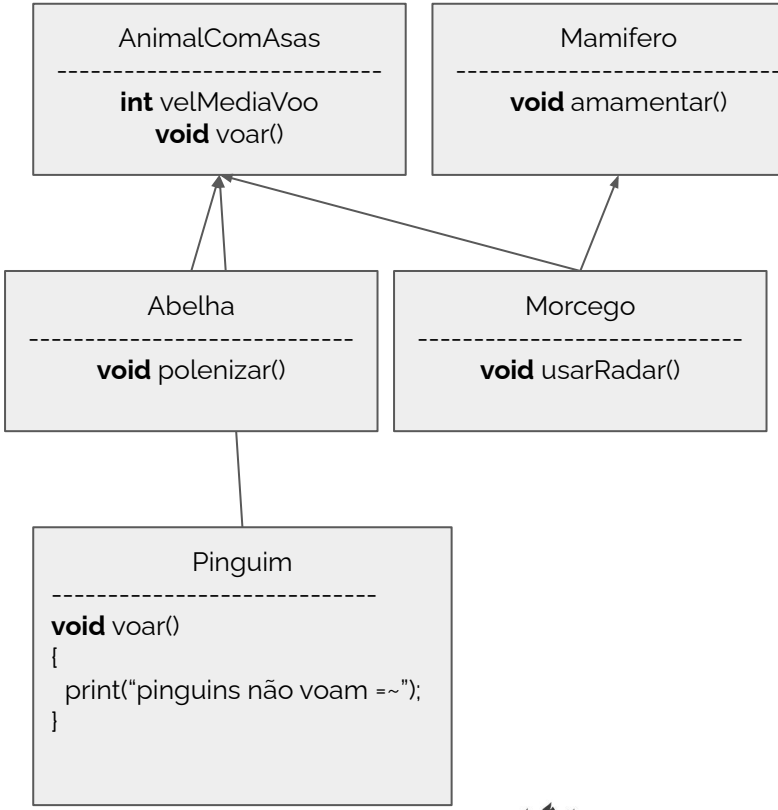
Uma **superclasse indireta** é qualquer classe acima da superclasse direta na **hierarquia de classes**, que define os relacionamentos de herança entre classes

No Java, a hierarquia de classes inicia com a classe **Object**, da qual toda classe em Java direta ou indiretamente **estende** (ou “herda de”)

Herança: Única x Múltipla

Herança única é aquela em que cada classe é derivada **exatamente de uma** superclasse direta

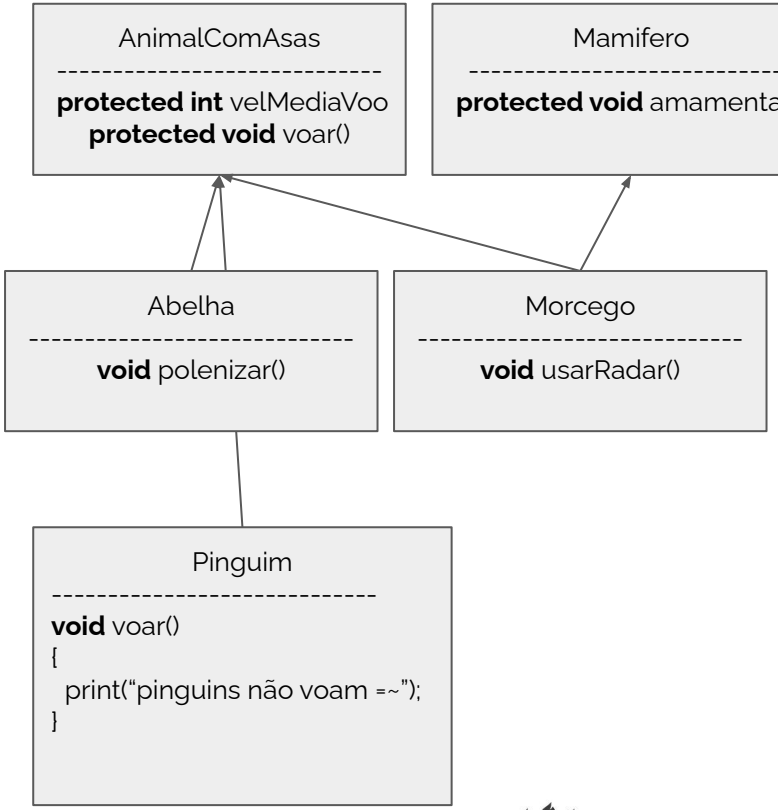
Ao contrário de C++, o Java não suporta **herança múltipla**, que ocorre quando uma classe é derivada de **mais de uma** superclasse direta



Modificadores de Acesso: Public x Private x Protected

Tudo em uma classe que consta como **private** é inacessível para outras classes, até aquelas que a herdam

Os membros **protected** de uma superclasse podem ser acessados por subclasses



Herança: Construtores em Subclasses

Instanciar um objeto de uma subclasse inicia uma **cadeia de chamadas do construtor** na qual ele, antes de realizar suas tarefas,

- usa *explicitamente* **super** para chamar um dos construtores em sua superclasse direta,
- ou então para chamar *implicitamente* o **construtor padrão** ou sem argumentos da superclasse.

Exercício

Inheritance in Java - GeeksforGeeks

```java
class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

Console:

```
Class A
Class B
Class A
Class C
Class A
Class D
```

Exercício

```java
class A {
    public void print_() { System.out.println("Class A"); }
}

class B extends A {
    public void print_() { System.out.println("Class B"); }
}

class C extends A {
    public void print_() { System.out.println("Class C"); }
}

class D extends A {
    public void print_() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_();

        C obj_C = new C();
        obj_C.print_();

        D obj_D = new D();
        obj_D.print_();

    }
}
```

Inheritance in Java - GeeksforGeeks

Console:

```
Class B
Class C
Class D
```

Polimorfismo

O que significa polimorfismo?

Várias formas

Polimorfismo

O polimorfismo permite

"**programar no geral**"

em vez de

"**programar no específico**"

Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

Qualquer operação (por exemplo, calcular o perímetro ou a área) que pode ser realizada em um objeto Quadrilátero **também pode ser realizada** em um objeto Retângulo

Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

```java
abstract class Forma2D {
    public double area;
    abstract public void calcularArea();
}

class Retangulo extends Forma2D {
    public double altura;
    public double base;

    public Retangulo(double altura, double base)
    {
        this.altura = altura;
        this.base = base;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = altura * base;
    }

    public String toString(){
        return "Retangulo de altura: " + altura + " e base: " + base;
    }
}
```

```java
class Circulo extends Forma2D {
    public double raio;

    public Circulo(double raio)
    {
        this.raio = raio;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = Math.PI * Math.pow(raio, b: 2);
    }

    public String toString(){
        return "Criculo de raio: " + raio;
    }
}
```

```java
public static void main(String[] args) {

    Retangulo r1 = new Retangulo(altura: 1, base: 2);
    Retangulo r2 = new Retangulo(altura: 3, base: 5);
    Circulo c1 = new Circulo(raio: 7);
    Circulo c2 = new Circulo(raio: 9);

    ArrayList<Forma2D> formas = new ArrayList<Forma2D>();
    formas.add(r1);
    formas.add(r2);
    formas.add(c1);
    formas.add(c2);

    for (Forma2D forma : formas) {
        System.out.println(forma);
        System.out.println(">> a área dessa forma é: " + forma.area + "\n");
    }

    System.out.println(r1);
    System.out.println(r2);
    System.out.println(c1);
    System.out.println(c2);
```

Conceito de Hoje

Conceito de Hoje

Arquitetura MVC: interface gráfica, componentes visuais e linguagens de marcação

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

Interface Gráfica do Usuário (Graphical User Interface ou GUI)

E se quisermos trabalhar com botões, sliders e outros componentes visuais?

Java -> Swing, JavaFX
C# -> UWP
Javascript -> CSS/HTML
...

Interface Gráfica do Usuário (Graphical User Interface ou GUI)

Como proceder com o código
somente quando o usuário clica em 1 botão?

Isso é equivalente a um escaneamento em Java?

E se tivermos dois ou mais botões?

Como descentralizar esse código para que
cada botão possa ser apertado de forma independente?

Interface Gráfica do Usuário (Graphical User Interface ou GUI)

Orientação a eventos pode ser uma resposta!

Cada pressionar de um botão gera um evento, que vai engatilhar a execução de um código em particular, de forma independente.

Apresentação

Definições

Exemplos Ilustrativos

Arquitetura MVC

- **Model:** What to render
- **View:** How to render
- **Controller:** Events, user input

[java - MVC pattern on Android - Stack Overflow]

Arquitetura MVC

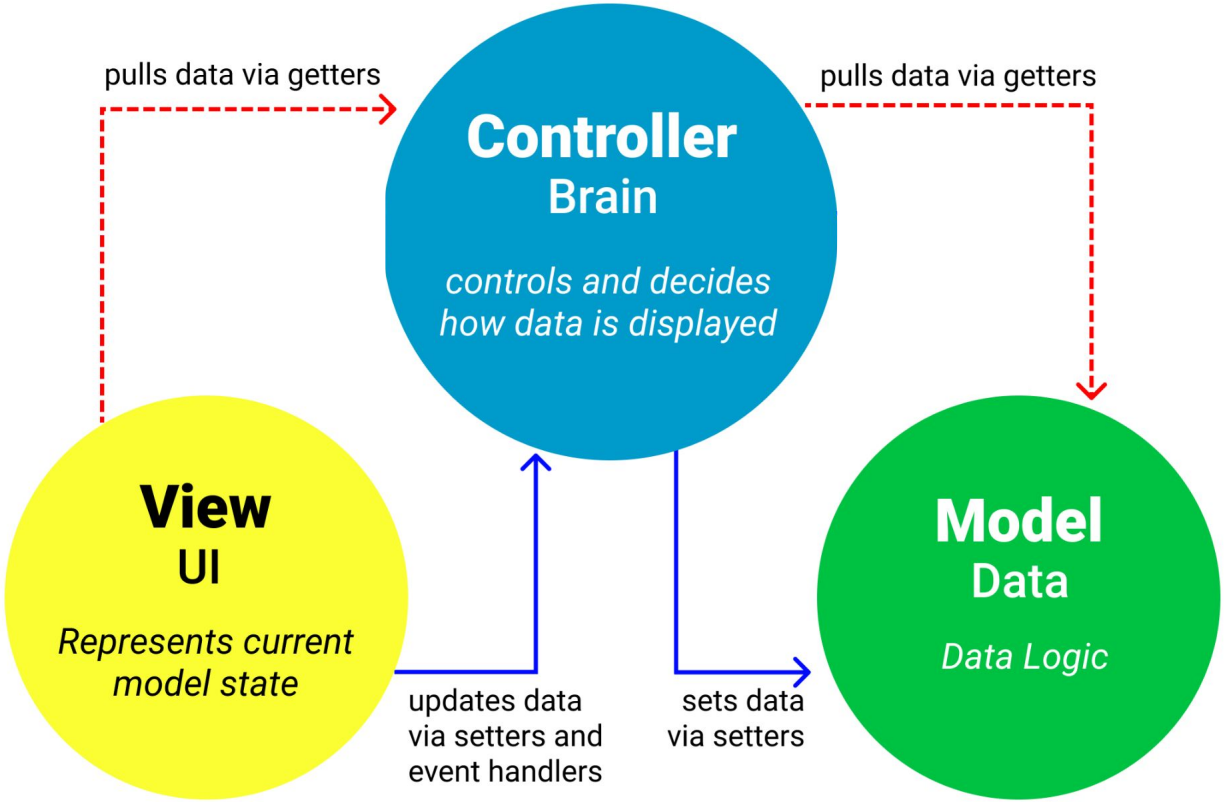
Arquitetura Model-View-Controller (MVC)

Aplicativos JavaFX em que a GUI é implementada como FXML seguem o **padrão de design Model-View-Controller** (**MVC**), que separa os dados de um aplicativo (contidos no **modelo**), de um lado, e, de outro, a GUI do aplicativo (a **visualização**) e a lógica de processamento do aplicativo (o **controlador**).

O controlador implementa a lógica para processar entradas do usuário. O modelo contém dados do aplicativo e a visualização apresenta os dados armazenados no modelo. Quando um usuário fornece alguma entrada, o controlador modifica o modelo com a entrada dada. No **Tip Calculator**, o modelo é o valor da conta, a gorjeta e o total. Quando o modelo muda, o controlador atualiza a visualização para apresentar os dados alterados.

Em um aplicativo JavaFX FXML, você define as rotinas de tratamento de evento do aplicativo em uma **classe de controladores**. A classe de controladores define as variáveis de instância para interagir com os controles programaticamente, bem como os métodos de tratamento de evento. A classe de controladores também pode declarar variáveis de instância adicionais, variáveis e métodos `static` que suportam a operação do aplicativo. Em um aplicativo simples como o **Tip Calculator**, o modelo e o controlador são muitas vezes combinados em uma única classe, como faremos nesse exemplo.

MVC Architecture Pattern



The Model View Controller Pattern – MVC Architecture and Frameworks Explained (freecodecamp.org)

Setup do JavaFX e do SceneBuilder

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Mini-Projeto 9

Interface Gráfica

- Faça o setup do JavaFX e do SceneBuilder como no vídeo mostrado na aula
- Crie uma janela com interface gráfica no SceneBuilder, salve o arquivo **.fxml** na pasta src do seu projeto (aqui você já pode fazer um commit)
    . Sua interface deve ter um **espaço** para mostrar o **texto dos capítulos**
    . Um outro espaço para mostrar as **imagens ASCII**
    . Também deve ter um **espaço para botões** com os textos das **escolhas**
        > Os botões podem já ser criados na interface
        > Ou criados via código java, e posicionados na interface
- Crie uma **classe controladora** para o arquivo **.fxml**
  (instruções para isso constam perto do final do vídeo)
    . A classe controladora deve **capturar os eventos** de clicar nos botões
    . Na sequência, atualize o capítulo mostrado na interface

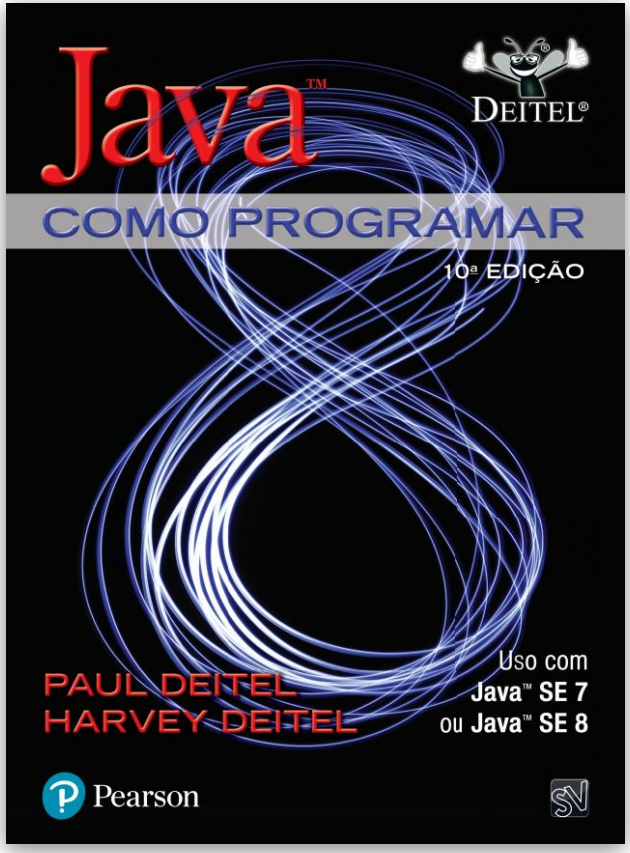
Dica: uma forma de realizar a atividade é substituir todas as chamadas sobre o Scanner que acessa o console e o System.out.println por chamadas do controlador criado

Mini-Projeto 10

Refatoração do código.

Padronize seu código a partir das seções especificadas do guia de estilo de Java do Google (https://google.github.io/styleguide/javaguide.html)

- Seção 2.1. Nomes dos arquivos fonte (.java e .fxml)
- Seção 4.2. Indentação de blocos.
- Seção 4.4. Limite da coluna
- Seção 5. Nomeação.
- Seção 7.3. Documentação em Java.

Referências

Java - Como Programar (10a Edição)